BÁO CÁO KỸ THUẬT: PHÂN VÙNG BẢN ĐỒ NHÀ KHO (WAREHOUSE SEGMENTATION) VÀ ĐIỀU HƯỚNG BẰNG GIỌNG NÓI TRỰC TIẾP QUA MICROPHONE VỚI NAV2

Họ và tên: Nguyễn Quang Tuân | **MSSV:** 23020767 | **GVHD:** PGS.TS. Hoàng Văn Xiêm, CN. Nguyễn Quốc Bảo

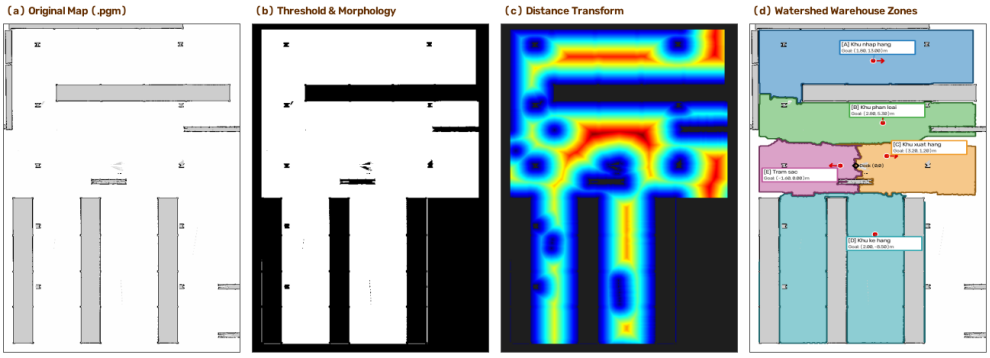
Nền tảng: ROS 2 Humble, Gazebo Fortress, OpenCV, Nav2, Faster-Whisper & Live Microphone STT | **GitHub:** https://github.com/Tuan404cl/Turtlebot_sim_test4

1 Phương Pháp Phân Vùng Bản Đồ Nhà Kho (Map Segmentation) & Gắn Nhãn Ngữ Nghĩa

1.1 Quy trình xử lý ảnh Occupancy Grid 2D của Nhà kho bằng OpenCV

Bản đồ lưới chiếm dụng 2D của nhà kho (`maps/map.pgm` và `maps/map.yaml`) có kích thước $W \times H = 603 \times 1003$ pixels, độ phân giải $r = 0.05$ m/pixel và gốc tọa độ $(x_0, y_0) = (-15.1, -25.1)$ m. Module `map_segmentation.py` thực hiện chuỗi 4 bước xử lý ảnh tự động để phân chia toàn bộ không gian bên trong nhà kho ($x \in [-10.0, 12.5]$ m, $y \in [-22.1, 16.8]$ m) thành 05 khu vực vận hành kho vận:

1. **Phân ngưỡng nhị phân (Binary Thresholding):** Tách vùng không gian trống di chuyển được ($I(u,v) \geq 250 \rightarrow 255$) khỏi tường bao/vách kệ ($I(u,v) = 0$) và vùng lõi kệ hàng kín ($I(u,v) = 205$).
2. **Lọc hình thái học (Morphological Filtering):** Áp dụng phép mở (`MORPH_OPEN`, kernel Ellipse 3×3) và phép đóng (`MORPH_CLOSE`, kernel Rect 5×5) nhằm khử nhiễu điểm quét LiDAR và làm liền mạch đường biên của dãy kệ ngang phía Bắc và 3 dãy kệ dọc phía Nam.
3. **Biến đổi khoảng cách Euclid (Distance Transform):** Tính trường khoảng cách $D(u,v) = \text{dist}(u,v) \times r$ từ mỗi điểm trống đến tường hoặc kệ hàng gần nhất bằng `cv2.distanceTransform(DIST_L2)`, đảm bảo tọa độ đích của từng khu vực luôn nằm tại vùng thông thoáng cách xa vật cản ($D(u,v) \geq 1.70$ m).
4. **Phân vùng Watershed có điều khiển hạt giống (Marker-Controlled Watershed):** Khởi tạo các vùng hạt giống tại 05 phân khu chức năng của nhà kho và áp dụng `cv2.watershed` trên phổ gradient đảo của trường khoảng cách để xác định ranh giới tự nhiên tại các lối giao.



Hình 1: Quy trình 4 bước xử lý ảnh phân vùng bản đồ Nhà kho (`map.pgm`) và gắn nhãn 05 khu vực vận hành `A, B, C, D, E`.

1.2 Quy đổi tọa độ Pixel sang ROS Map và lưu trữ dữ liệu khu vực nhà kho

Với gốc ảnh $(u = 0, v = 0)$ ở góc trên bên trái và gốc bản đồ ROS (x_0, y_0) ở góc dưới bên trái, công thức chuyển đổi song ánh giữa tọa độ pixel (u, v) và tọa độ thực (x, y) trong hệ quy chiếu `map` là:

$$x = x_0 + (u + 0.5) \times r, \qquad y = y_0 + (H - 1 - v + 0.5) \times r \tag{1}$$

Toàn bộ thông tin 05 khu vực được xuất ra `config/semantic_regions.json` và `config/semantic_regions.yaml`, đồng thời hiển thị trực tiếp dưới dạng nhãn 3D `MarkerArray` (`/semantic_regions_markers`) trên giao diện RViz2.

Bảng 1: Bảng thông số 05 khu vực chức năng nhà kho trong `semantic_regions.json` và `semantic_regions.yaml`.

ID	Tên khu vực nhà kho	Từ khóa giọng nói nhận diện (Aliases)	Pixel (u, v)	Tọa độ ROS (x, y, yaw)	Diện tích	Clearance
A	Khu nhập hàng	`khu vực a, khu nhập hàng, kho nhập hàng`	$(338, 240)$	$(1.80, 13.00, 0.00)$	155.07 m^2	3.00 m
B	Khu phân loại	`khu vực b, khu phân loại, kiểm hàng`	$(358, 394)$	$(2.80, 5.30, 1.57)$	105.86 m^2	2.70 m
C	Khu xuất hàng	`khu vực c, khu xuất hàng, khu đóng gói`	$(366, 476)$	$(3.20, 1.20, 0.00)$	71.82 m^2	1.93 m
D	Khu kệ hàng	`khu vực d, khu kệ hàng, kho lưu trữ`	$(342, 670)$	$(2.00, -8.50, -1.57)$	203.32 m^2	2.90 m
E	Trạm sạc	`khu vực e, trạm sạc, khu điều phối`	$(270, 500)$	$(-1.60, 0.00, 3.14)$	60.98 m^2	1.70 m

2 Nhận Dạng Giọng Nói Trực Tiếp Từ Microphone & Điều Hướng Đa Chặng Với Nav2

2.1 Thu âm trực tiếp từ Microphone & Nhận dạng tiếng Việt (voice_command_node.py)

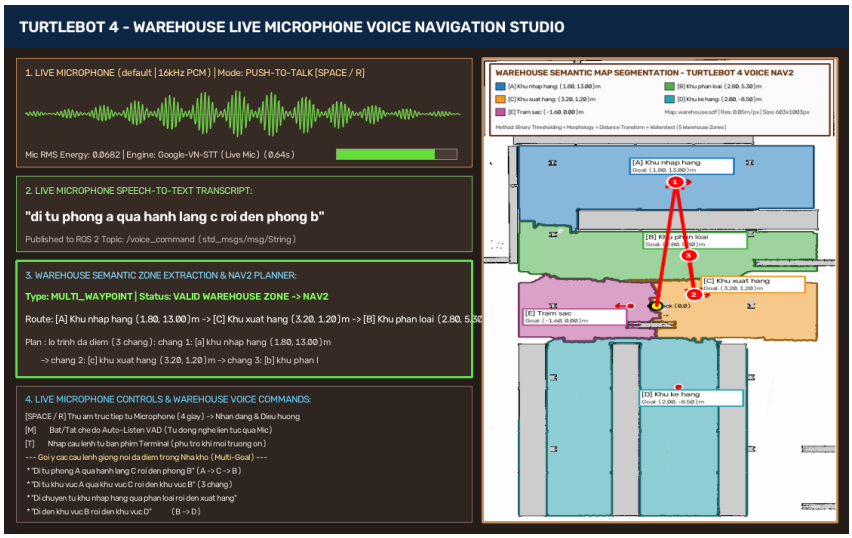
Thay vì sử dụng các file âm thanh tạo sẵn, node `voice_command_node` thu nhận tín hiệu giọng nói **trực tiếp từ Microphone phần cứng** trên máy tính ở định dạng chuẩn PCM 16kHz, 16-bit Mono qua giao tiếp ALSA/PulseAudio (`arecord`). Khi khởi động, hệ thống tự động kích hoạt và chuẩn hóa mức âm lượng thu (90%) cho Digital Microphone tích hợp (`sof-hda-dsp DMIC`). Hệ thống hỗ trợ hai chế độ thu trực tiếp: (1) **Push-to-Talk** (nhấn phím `SPACE` hoặc `R` để thu câu lệnh 4 giây) và (2) **Auto-Listen VAD** (nhấn phím `M` để tự động lắng nghe liên tục khi năng lượng hiệu dụng RMS ≥ 0.012). Luồng âm thanh thu từ Microphone được chuyển thành văn bản tiếng Việt qua **SpeechRecognition** (`vi-VN`) kết hợp mô hình AI Offline **Faster-Whisper** (`base-int8`) và publish lên topic `/voice_command`.

2.2 Giải thuật phân tích câu lệnh nhà kho & điều hướng đa điểm (voice_navigation_node.py)

Node `voice_navigation_node` chuẩn hóa văn bản tiếng Việt và xử lý 3 lớp kịch bản điều hướng kho vận:

- **Điều hướng đơn đích (`SINGLE_GOAL`):** Với các câu lệnh như *"Đi đến khu vực A"* hoặc *"Đi đến khu nhập hàng"*, bộ phân tích ánh xạ tên khu vực sang tọa độ đích (x, y, yaw) trong `semantic_regions.json` và gửi yêu cầu tới Nav2.
- **Điều hướng nhiều điểm đến (`MULTI_WAYPOINT`):** Xử lý các câu lệnh phức hợp liên hoàn từ 2 đến N điểm như *"Đi từ phòng A qua hành lang C rồi đến phòng B"* hoặc *"qua A qua B qua C qua D rồi qua E"*. Bộ phân tích bóc tách chuỗi các chặng: [Xuất phát] → [Trung gian 1, 2...] → [Đích cuối]. Hệ thống tự động kiểm tra vị trí AMCL: nếu robot chưa ở điểm xuất phát sẽ qua điểm xuất phát trước, sau đó tuần tự điều hướng qua các trạm dừng. Toàn bộ lộ trình dự kiến được publish tức thời lên RViz2 dạng `nav_msgs/msg/Path` (`/voice_nav/planned_path`) và hiển thị số thứ tự chặng (1, 2, 3...) trên cả RViz2 lẫn cửa sổ bản đồ riêng.
- **Xử lý an toàn khi khu vực không tồn tại (`INVALID_REGION`):** Khi câu lệnh chứa tên vùng không thuộc nhà kho (ví dụ: *"Đi đến khu vực Z"* hoặc *"Đi từ phòng A qua phòng Z rồi đến phòng B"*), hệ thống trả về `INVALID_REGION`, giữ robot dừng an toàn và cảnh báo rõ tên vùng sai.

3 Kết Quả Thử Nghiệm Thực Tế (Experimental Results)



Hình 2: Giao diện *Warehouse Live Microphone Voice Navigation Studio* khi thu câu lệnh đa điểm trực tiếp từ Microphone: hiển thị sóng âm thời gian thực, phân tích lộ trình đa trạm tuần tự và vẽ đường đi dự kiến.

Bảng 2: Kết quả thực nghiệm nhận dạng câu lệnh giọng nói trực tiếp qua Microphone và điều hướng đa điểm với Nav2.

TT	Câu lệnh giọng nói qua Microphone	Loại lệnh	Lộ trình Waypoints	Hành vi thực thi trên hệ thống Nav2
1	*"qua A qua B qua C qua D rồi qua E"*	`MULTI_WAYPOINT`	[A → B → C → D → E]	Tuần tự 5 trạm: Nhập hàng → Phân loại → Xuất hàng → Kệ hàng → Sạc
2	*"Đi từ phòng A qua hành lang C rồi đến phòng B"*	`MULTI_WAYPOINT`	[A → C → B]	Điều hướng 3 chặng qua A, C rồi đến B, vẽ lộ trình lên RViz2
3	*"Đi chuyển từ khu vực B đến khu vực C"*	`MULTI_WAYPOINT`	[B → C]	Đi qua Khu phân loại (B) trước khi tới Khu xuất hàng (C)
4	*"Đi đến khu nhập hàng"*	`SINGLE_GOAL`	[A]	Điều hướng trực tiếp tới Khu nhập hàng phía Bắc (1.80, 13.00) m
5	*"Quay về trạm sạc"*	`SINGLE_GOAL`	[E]	Trở về Dock sạc tự động tại vị trí trung tâm (−1.60, 0.00) m
6	*"qua A qua B qua C qua Z rồi qua E"*	`INVALID_REGION`	∅	Phát hiện vùng `Z` không tồn tại, từ chối toàn bộ lộ trình, cảnh báo an toàn

Kết luận: Hệ thống hoàn thiện khả năng nhận dạng giọng nói từ microphone thời gian thực, phân tích chuỗi lệnh phức tạp với N điểm dừng liên hoàn (*"qua A qua B qua C qua D rồi qua E"*), trực quan hóa đường đi dự kiến trên cả RViz2 (`/voice_nav/planned_path`) lẫn cửa sổ bản đồ riêng, đảm bảo robot TurtleBot 4 vận hành an toàn và linh hoạt trong kho vận.